Architectural Framework for Right-Fitting LLM Models to a Multi-Agent SDLC Workflow on 48GB Apple Silicon

The landscape of software development has been fundamentally altered by the advent of agentic workflows, particularly those that orchestrate the entire Software Development Lifecycle (SDLC) through a hierarchy of specialized Large Language Model (LLM) agents. Operating such a system, comprised of 3 orchestrator rules and 17 worker subagents, presents a significant challenge in balancing computational efficiency, reasoning depth, and operational cost. As of March 2026, the transition from purely cloud-based commercial models to a hybrid architecture—leveraging the unique unified memory architecture (UMA) of Apple Silicon and the burgeoning ecosystem of free cloud API tiers—has become a viable path for senior engineers and research teams.[1] This report provides an exhaustive analysis of how to optimize a sequential, multi-agent SDLC workflow on a hardware profile centered on a 48GB unified memory Mac, utilizing the MLX framework for local inference and a tiered escalation protocol for quality assurance.[3]

Hardware Dynamics and the Constraint of Unified Memory

Understanding the physical limitations and advantages of the Apple Silicon M-series platform is foundational to model selection. Unlike traditional PC architectures that separate system RAM from Video RAM (VRAM), Apple's unified memory allows the CPU, GPU, and Neural Engine to access the same high-bandwidth pool.[2] In a 48GB configuration, the effective budget for LLM residency is approximately 38-40GB, with 8-10GB typically reserved for macOS system processes and the overhead of modern Integrated Development Environments (IDEs) like Cursor and Roo-Code.[4]

The primary bottleneck in local LLM inference is memory bandwidth rather than raw compute cycles. Each generated token requires reading the model's weights from memory, a process that scales linearly with the model's parameter count and the bit-depth of its quantization.[2] For a system aiming to maximize the utility of local weights, the use of 4-bit quantization (Q4_K_M or INT4) is non-negotiable, as it reduces memory requirements by approximately 75% compared to standard 16-bit precision (FP16) while retaining 97-99% of the original model's accuracy.[8]

Furthermore, the implementation of the MLX framework, developed by Apple's machine

learning research team, exploits UMA directly to eliminate PCIe bus transfer overhead, providing significantly higher throughput on Mac hardware compared to cross-platform solutions.[1] Recent benchmarks on M4 and M5 series chips indicate that MLX often achieves 20% to 80% higher throughput than llama.cpp on identical models, making it the preferred runtime for persistent agentic workflows.[9]

Section 1: Local Model Technical Specifications Under Real-World Conditions

To right-fit the 17 worker agents, an audit of available open-weights models is required. These models fall into three architectural categories: dense models, Mixture-of-Experts (MoE) models, and multimodal/vision-capable models.

Mixture-of-Experts (MoE) and Parametric Efficiency

MoE models represent the current state-of-the-art for local deployment on 48GB hardware. By activating only a fraction of their total parameters for any given token—for instance, 3 billion active parameters out of an 80 billion parameter total—these models deliver the reasoning capabilities of a large model with the inference speed of a small one.[4] However, memory requirements are governed by total parameter count rather than active parameters; the full weight set must be resident in memory during inference.[11]

Model Component	Architecture	Quantization	Weight Footprint (GB)	32K KV Cache (GB)	128K KV Cache (GB)
Qwen3-Coder-Next	80B MoE (3B active)	4-bit (Q4_K_XL)	40.8 - 41.0	5.0 - 8.0	20.0 - 32.0
Qwen3-30B-A3B	30B MoE (3B active)	4-bit (Q4_K_M)	15.6 - 16.0	3.0 - 4.0	12.0 - 16.0
DeepSeek-R1-Distill-32B	32B Dense	4-bit (Q4_K_M)	19.8 - 21.0	4.0 - 6.0	16.0 - 24.0

GPT-OSS-20B	21B MoE (4B active)	4-bit (mxfp4)	11.0 - 12.0	2.0 - 3.0	8.0 - 12.0

Data indicates that MoE models like the Qwen3 series utilize a hybrid attention mechanism (Gated DeltaNet + Gated Attention) which scales linearly or near-linearly for longer contexts, providing 10x higher throughput for contexts exceeding 32K tokens compared to previous architectures.[10] This is critical for agents like the Story Decomposer or Plan Validator, which must ingest large documentation sets.

Key-Value (KV) Cache Management

The KV cache is the most dynamic and often overlooked consumer of VRAM. For a 48GB Mac, managing the cache budget is paramount. At context windows of 128K tokens, the cache can easily exceed the size of the model weights if stored in FP16 precision.[14] Strategies to mitigate this include KV cache quantization (4-bit or 8-bit), which halves or quarters the memory per token with minimal quality loss.[16] Research confirms that on Apple Silicon, 4-bit (kv4) should be the default for long-context sessions, as it provides 3.2x more capacity with negligible performance impact.[18]

Section 2: Mapping Open-Source LLMs to SDLC Agent Tiers

The 17 SDLC agents can be categorized into six functional tiers based on their reasoning requirements, output length, and domain knowledge. Right-fitting involves assigning a local model that meets the "good enough" quality threshold for each tier to prevent unnecessary cloud escalation.

Tier 1: Deep Planning

Agents: PRD Agent, System Architecture Agent, Story Decomposer. **Needs:** Exceptional reasoning, adherence to complex 14-section templates, and the ability to process 32-64K context tokens.[19] **Recommended Local Model:** DeepSeek-R1-Distill-Qwen-32B (4-bit). **Justification:** This model, distilled from the 671B DeepSeek-R1, delivers o1-level reasoning.[20] It excels in math and logical deduction, which is essential for requirements sparring and architectural topology design.[20] At 21GB (weights) + 12GB (64K context), it fits comfortably within the 48GB budget.[21]

Tier 2: Domain Technical Planning

Agents: HLD Agent, API Design Agent, Data Architecture Agent, Security Agent, DevOps

Agent, Testing Strategy Agent. **Needs:** Technical standards knowledge (OpenAPI, WCAG, Threat Modeling) and structured output generation.[24] **Recommended Local Model:** Qwen3-30B-A3B (4-bit). **Justification:** This MoE model offers balanced performance and technical accuracy across coding and design domains.[19] Its 3B active parameters allow for rapid sequential dispatches during story-level planning loops.[10]

Tier 3: Code Generation

Agent: Implementer. **Needs:** Highest SWE-bench scores, reliable tool use, and 64-128K context for project-scale implementation.[4] **Recommended Local Model:** Qwen3-Coder-Next (4-bit). **Justification:** This 80B MoE model is the "speed king" for coding on local hardware, matching Claude Sonnet 4.5 on several benchmarks.[4] It takes approximately 41GB of RAM, making it a "Marginal Fit" that requires KV cache quantization to 4-bit to support 32K+ context.[4]

Tier 4: Fast Verification

Agents: Code Reviewer, QA Verifier, Acceptance Validator, Project Research. **Needs:** Sub-3-second latency, structured comparison capability, and evidence-based pass/fail decisions.[27] **Recommended Local Model:** GPT-OSS-20B (4-bit) or Gemma 3 12B. **Justification:** These smaller models can stay resident in memory alongside a primary agent or be loaded in seconds.[19] They are proficient at following checklists and comparing code diffs against plans.[19]

Tier 5: Cross-Plan Validation

Agent: Plan Validator. **Needs:** Massive context (128K-1M) to cross-reference an entire project directory.[30] **Recommendation:** Escalate to Gemini 2.5 Pro (Free Cloud Tier). **Justification:** No local model on 48GB can ingest 80K tokens while maintaining the necessary reasoning depth to detect cross-document contract violations.[30] Gemini 2.5 Pro's 1M context window is uniquely suited for this task.[30]

Tier 6: Multimodal Analysis

Agent: Design/UI-UX Agent. **Needs:** Vision capability for screenshot review and accessibility audits.[34] **Recommended Local Model:** Llama-3.2-11B-Vision-Instruct or Qwen3-VL. **Justification:** Native vision-language models can identify color contrast issues and structural violations in heading hierarchies.[35]

Section 3: Free Cloud API Providers for SDLC Workloads

To supplement local inference and manage memory high-water marks, free cloud API tiers should be integrated as fallbacks or primary endpoints for high-latency tasks.

Provider	Available Models	Rate Limits (2026)	Strengths
Google AI Studio	Gemini 2.5 Pro, Flash, Lite	100 RPD (Pro), 250 RPD (Flash), 1000 RPD (Lite)	1M token context, multimodal [30]
Groq	Llama 3.3 70B, Qwen3-32B	1000 RPD, 30 RPM, 300K TPM	Sub-1s latency, high throughput [37]
Cerebras	GPT-OSS 120B, Llama 3.1 8B	14,400 RPD, 30 RPM, 1M Tokens/Hr	"World's fastest" inference (3000+ TPS) [39]
OpenRouter	Step 3.5 Flash, Trinity Large	200 RPD, 20 RPM (with $10 top-up)	Model diversity, unified API [41]

The practical daily throughput for a full SDLC session (~190 dispatches) can be handled entirely within these free tiers if load is balanced across providers.[30] Specifically, Cerebras and Groq are ideal for Tier 4 verification, while Google AI Studio is the anchor for Tier 1 and Tier 5 planning tasks that exceed local context capabilities.[30]

Section 4: Memory Budget Analysis for Sequential SDLC Workflow

A critical component of this optimization is the memory residency timeline. Because the SDLC workflow is sequential, only one heavy reasoning or implementation model needs to be loaded at a time.[44]

Planning Phase Timeline

In the 7-phase planning workflow, the memory footprint is relatively stable. Phase 1 through Phase 4 involves dispatches that typically require 16-32K context. Using the DeepSeek-R1-Distill-32B (21GB) leaves approximately 17-19GB for the KV cache and framework buffers.[21]

- **Loading overhead:** MLX lazy loading completes in under 30 seconds for a 32B model, making model swaps between planning phases efficient.[46]
- **Context dominance:** At 128K context, the KV cache (always FP16 by default in many

runners) will dominate memory bandwidth, slowing generation regardless of model quantization.[27]

Execution Dev Loop Analysis

The implementation loop (Implementer -> Code Reviewer -> QA) is the point of maximum memory stress.

- **Strategy A (Sequential Swap):** Loading the 41GB Qwen3-Coder-Next for implementation, then unloading it to load a 12GB verification model. This adds ~4 minutes of load time per task cycle.[11]
- **Strategy B (Dual Residency):** Keeping a Qwen3-Coder-30B (15.6GB) and a verification model like GPT-OSS-20B (11.0GB) resident simultaneously. Total weights = 26.6GB. This fits comfortably with 10-12GB available for KV caches, allowing near-zero swap latency.[19]
- **Strategy C (Hybrid Local+Cloud):** This is the **Performance Leader**. Keeping the largest local coder (Qwen3-Coder-Next, 41GB) resident and offloading all Code Reviewer and QA tasks to Groq or Cerebras APIs. This allows the local hardware to devote its entire memory pool to the Implementer's long-context repository processing.[48]

Section 5: Escalation Protocols and Gate Validation Triggers

The system relies on a tiered escalation to maintain quality while minimizing cost. Triggers are integrated with existing phase validation gates.

Planning Phase Triggers

1. **PRD Agent:** If the PRD fails the 8-dimension validation gate twice, it indicates a semantic failure in requirement challenging. **Escalate to:** Claude 4 Opus.
2. **System Architecture:** If the architecture document has unresolved cross-references after one validation attempt, it indicates a structural reasoning failure. **Escalate to:** Claude 4 Opus.
3. **Per-Story Design:** If the Plan Validator flags more than three consistency issues across a story's HLD/API/Data/Security documents, the story-loop is reset on the cloud. **Escalate to:** Claude 4 Sonnet.

Execution Phase Triggers

1. **Implementer:** If the Code Reviewer rejects 2 out of 3 attempts for a single implementation unit, the task is marked as "Complex Logic Failure." **Escalate to:** Claude 4 Opus or GPT-5.3 Codex.
2. **Verification False-Pass:** If the QA Verifier finds bugs that the Code Reviewer missed, the review process is deemed unreliable. **Escalate review to:** Claude 4 Sonnet.

3. **Acceptance Validation:** Story artifacts for security-critical domains (payments, auth) are routed to commercial models by default for a final compliance check.[34]

Complexity-Based Routing Matrix

Agents are classified by task complexity before dispatch:

- **Simple:** Formatting, documentation, boilerplates. (Local/Free Cloud only).
- **Moderate:** Refactoring, schema updates, contract generation. (Local first, escalate on gate fail).
- **Complex:** Algorithm design, state machine logic, cryptographic implementation. (Free Cloud primary, escalate on gate fail).
- **Critical:** Security-sensitive code, final full-chain SDLC validation. (Always Commercial).

Section 6: Token and Cost Savings Model

Current baseline pricing (Projected March 2026):

- **Claude 4 Opus:** $15.00 Input / $75.00 Output per 1M tokens.[51]
- **Claude 4 Sonnet:** $3.00 Input / $15.00 Output per 1M tokens.[53]
- **Claude 4 Haiku:** $1.00 Input / $5.00 Output per 1M tokens.[53]

Project Cost Projection (Standard 5-Story Project)

Phase	Dispatches	Avg Tokens (K)	Baseline Cost (Sonnet/Opus)	Optimized Cost (Local/Free)	Savings (%)
Planning	33	15K In / 10K Out	~$240.00	~$12.00 (Escalations)	95%
Execution	110	20K In / 8K Out	~$680.00	~$45.00 (Escalations)	93%
Total	143		~$920.00	~$57.00	**~94%**

Calculations account for a 15% escalation rate where local models fail gates and require commercial model re-runs.[55] Strategy 3 (Hybrid-Optimal) provides the best ROI by utilizing Cerebras for zero-cost, high-volume verification tasks and Google AI Studio for long-context reasoning, reserving commercial tokens for only the most difficult 5-10% of tasks.[30]

Section 7: Comprehensive Model Assignment Matrix

The following table provides the definitive assignment for all 17 agents in the system.

Agent Name	slug	Local Primary Model	RAM	Fallback Cloud	Escalation Target	Trigger
PRD Agent	planner-prd	DS-R1-Distill-32B	21G	Gemini 2.5 Pro	Claude 4 Opus	Gate Fail 2x
Architecture Agent	planner-architecture	DS-R1-Distill-32B	21G	Gemini 2.5 Pro	Claude 4 Opus	Gate Fail 1x
Story Decomposer	planner-stories	Qwen3-30B-A3B	16G	Qwen3-32B (Groq)	Claude 4 Sonnet	Dep. Loop
HLD Agent	planner-hld	Qwen3-30B-A3B	16G	Qwen3-32B (Groq)	Claude 4 Sonnet	Schema Fail
Security Agent	planner-security	Qwen3.5-27B	15G	GPT-OSS-120B (Cere)	Claude 4 Sonnet	Critical Risk
API Design Agent	planner-api	Qwen3-30B-A3B	16G	Gemini 2.5 Flash	Claude 4 Sonnet	Spec Invalid

Data Architecture	planner-data	Qwen3-30B-A3B	16G	Gemini 2.5 Flash	Claude 4 Sonnet	Mig. Error
DevOps Agent	planner-devops	Qwen3.5-27B	15G	Llama 3.3 70B (Groq)	Claude 4 Sonnet	Pipeline Fail
Design/UI-UX	planner-design	Llama 3.2 11B Vis	22G	Gemini 2.5 Pro	GPT-4o	WCAG < 75%
Testing Strategy	planner-testing	Qwen3.5-27B	15G	Llama 3.3 70B (Groq)	Claude 4 Sonnet	Low Coverage
Plan Validator	plan-validator	Gemini (Cloud)	0G	Gemini 2.5 Pro	Claude 4 Opus	High Complexity
Implementer	implementer	Qwen3-Coder-Next	41G	Qwen3-Coder (OR)	Claude 4 Opus	Reject 2x
Code Reviewer	code-reviewer	GPT-OSS-20B	11G	Llama 3.1 8B (Groq)	Claude 4 Haiku	False Pass
QA Verifier	qa-verifier	GPT-OSS-20B	11G	Llama 3.1 8B (Groq)	Claude 4 Haiku	Test Timeout
Acceptance Validator	accept-validator	GPT-OSS-20B	11G	Llama 3.1 8B (Groq)	Claude 4 Haiku	Logic Gap

Project Research	project-research	Qwen3.5-9B	5G	Gemini 2.5 Flash	Claude 4 Haiku	Low Evidence
Documentation Writer	doc-writer	Qwen3.5-9B	5G	Gemini 2.5 Flash	Claude 4 Haiku	Format Error

Section 8: Memory Scheduling Timeline

This timeline maps the VRAM watermarks across the project lifecycle, ensuring no OOM (Out of Memory) crashes occur on the 48GB system.

Planning Phase 1-2 (Requirements & Architecture)

- **Model:** DeepSeek-R1-Distill-32B (21GB).[23]
- **Cache:** 64K window (8GB).[7]
- **Status: Comfortable (29GB / 48GB)**.
- **Action:** Stay resident.

Planning Phase 3 (Story Loop)

- **Model:** Qwen3-30B-A3B (16GB).[24]
- **Cache:** 128K window (16GB).[7]
- **Status: Optimal (32GB / 48GB)**.
- **Action:** Rapid sequential dispatches of domain agents.

Planning Phase 5 (Full Validation)

- **Model:** Gemini 2.5 Pro (Cloud Primary).
- **Status: Minimal Local Footprint (0GB + IDE)**.
- **Action:** Offload to free cloud for 1M context analysis.[30]

Execution Phase 2 (Implementation Loop - Strategy C)

- **Primary Local Model:** Qwen3-Coder-Next (41GB).[4]
- **Cache:** 32K window (6GB, 4-bit KV).[16]
- **Cloud Verification:** (Groq/Cerebras APIs).
- **Status: Peak Pressure (47GB / 48GB)**.
- **Action:** Use --cache-memory-mb 6000 to prevent memory expansion crashes.[17]

Execution Phase 5 (Documentation)

- **Model:** Qwen3.5-9B (5GB).[19]
- **Status: Minimal (5GB / 48GB)**.
- **Action:** Zero loading tax, high speed.

Section 9: IDE Integration and Tactical Configuration

Connecting Cursor and Roo-Code to this three-tier system requires a robust serving layer and specific configuration files.

9.1 Model Serving with LM Studio and MLX

LM Studio (v0.4.6) is the recommended backend due to its native MLX support and OpenAI-compatible API.[48]

- **Hot-Swapping:** Use client.llm.model() for specialist agents to trigger JIT (Just-In-Time) loading.
- **Residency:** Set ttl=0 for the PRD and Implementer agents to prevent eviction during long reasoning blocks.[48]
- **Continuous Batching:** Enable for the MLX engine to support parallel verify tasks in Strategy C.[9]

9.2 Cursor Configuration

Cursor agents are configured via .cursor/agents/*.md with YAML frontmatter.

- **Routing:** Since Cursor doesn't support per-agent endpoints, use **LiteLLM** as a proxy server.[48] Point Cursor to localhost:8000.
- **Frontmatter:** Map the model key to specific LiteLLM routes.

YAML

```yaml
---
name: implementer
description: Writes code within architecture scope.
model: local-qwen3-coder-80b
---
```

9.3 Roo-Code Configuration

Roo-Code utilizes .roomodes for agent personas and Configuration Profiles for API connectivity.[60]

- **Profiles:** Create three profiles: Local-MLX, Cloud-Free-Groq, and Cloud-Free-Gemini.
- **Sticky Assignment:** Associate each agent mode with its corresponding profile. Roo-Code will automatically switch endpoints when delegating subtasks via the new_task tool.[45]

Section 10: Prioritized Implementation Roadmap

To achieve maximum savings with minimum operational risk, implement changes in the following sequence:

1. **Phase 1: Verification Migration (Days 1-3)**
   - Target: Code Reviewer, QA, Acceptance Validator.
   - Action: Route to Cerebras/Groq free tiers (Strategy C).
   - Risk: Zero. Verification is pass/fail based on provided plans.
   - Impact: 30% reduction in commercial token spend.
2. **Phase 2: Local Reasoning Integration (Days 4-7)**
   - Target: PRD Agent, Architecture Agent.
   - Action: Deploy DeepSeek-R1-Distill-32B via LM Studio.
   - Risk: High. Requires careful monitoring of gate validation failure rates.
   - Impact: Additional 20% savings.
3. **Phase 3: Technical Agent Rollout (Days 8-12)**
   - Target: HLD, API, Data, Security, DevOps agents.
   - Action: Deploy Qwen3-30B-A3B locally.
   - Impact: Full automation of Phase 3-4 planning at zero marginal cost.
4. **Phase 4: Coder Optimization (Days 13-21)**
   - Target: Implementer.
   - Action: Deploy Qwen3-Coder-Next with 4-bit KV cache quantization.
   - Impact: 40% reduction in overall cost. Final project costs drop to <$60.[4]

Conclusion

The optimization of a 20-component multi-agent SDLC workflow on 48GB Apple Silicon hardware requires a paradigm shift from monolithic cloud reliance to granular model right-fitting. By leveraging high-sparsity MoE models like Qwen3-Coder-Next for implementation and distilled reasoning models like DeepSeek-R1 for planning, developers can reclaim sovereignty over their development environments. Offloading verification to "super-fast" free cloud APIs like Cerebras ensures that local VRAM is preserved for the most complex reasoning tasks. This hybrid architecture not only yields net project savings of over 90% but also provides a more resilient, low-latency, and private workflow tailored to the specific strengths of the Apple Silicon ecosystem.[1]

Works cited

1. Local LLMs Apple Silicon Mac 2026 | M1 M2 M3 Guide - SitePoint, accessed March 15, 2026, https://www.sitepoint.com/local-llms-apple-silicon-mac-2026/

2. Local LLM Hardware Requirements: Mac vs PC 2026 - SitePoint, accessed March 15, 2026, https://www.sitepoint.com/local-llm-hardware-requirements-mac-vs-pc-2026/
3. Exploring LLMs with MLX and the Neural Accelerators in the M5 GPU, accessed March 15, 2026, https://machinelearning.apple.com/research/exploring-llms-mlx-m5
4. Qwen3-Coder-Next: The Complete 2026 Guide to Running Powerful AI Coding Agents Locally - Dev.to, accessed March 15, 2026, https://dev.to/sienna/qwen3-coder-next-the-complete-2026-guide-to-running-powerful-ai-coding-agents-locally-1k95
5. Apple has a sleeper advantage when it comes to local LLMs - XDA Developers, accessed March 15, 2026, https://www.xda-developers.com/apple-sleeper-advantage-local-llms/
6. Running DeepSeek R1 on Consumer GPUs: RTX 4090 vs M3 Max | SitePoint, accessed March 15, 2026, https://www.sitepoint.com/deepseek-r1-consumer-gpus-rtx4090-m3max/
7. Context Kills VRAM: How to Run LLMs on consumer GPUs | by Lyx | Medium, accessed March 15, 2026, https://medium.com/@lyx_62906/context-kills-vram-how-to-run-llms-on-consumer-gpus-a785e8035632
8. Quantization Explained: Q4_K_M vs AWQ vs FP16 for Local LLMs - SitePoint, accessed March 15, 2026, https://www.sitepoint.com/quantization-explained-q4km-vs-awq-vs-fp16-for-local-llms/
9. Native LLM and MLLM Inference at Scale on Apple Silicon - arXiv.org, accessed March 15, 2026, https://arxiv.org/html/2601.19139v2
10. Qwen3-Next-80B-A3B-Instruct: Efficient Long-Context AI | DigitalOcean, accessed March 15, 2026, https://www.digitalocean.com/community/tutorials/qwen3-next-80b-a3b-instruct-long-context-ai
11. Qwen/Qwen3-Next-80B-A3B-Instruct · How much GPU memory is needed for local deployment? - Hugging Face, accessed March 15, 2026, https://huggingface.co/Qwen/Qwen3-Next-80B-A3B-Instruct/discussions/7
12. GPU System Requirement Guide for Qwen 3.5 - ApX Machine Learning, accessed March 15, 2026, https://apxml.com/posts/qwen-3-5-system-requirement-vram-guide
13. Introduction - Qwen, accessed March 15, 2026, https://qwen.ai/blog?id=e34c4305036ce60d55a0791b170337c2b70ae51d
14. GPU Memory Requirements for LLMs: VRAM Calculator and Sizing Guide | Spheron Blog, accessed March 15, 2026, https://www.spheron.network/blog/gpu-memory-requirements-llm/
15. Accelerate Large-Scale LLM Inference and KV Cache Offload with CPU-GPU Memory Sharing | NVIDIA Technical Blog, accessed March 15, 2026, https://developer.nvidia.com/blog/accelerate-large-scale-llm-inference-and-kv-cache-offload-with-cpu-gpu-memory-sharing/

16. Context Size Frustration : r/LocalLLaMA - Reddit, accessed March 15, 2026, https://www.reddit.com/r/LocalLLaMA/comments/1r9pxxc/context_size_frustration/
17. KV cache Quantization. · waybarrios vllm-mlx · Discussion #17 - GitHub, accessed March 15, 2026, https://github.com/waybarrios/vllm-mlx/discussions/17
18. UMA-Native KV-Cache Benchmarks on M4 Pro 64GB – kv4 Outperforms Unquantized · ml-explore mlx · Discussion #3134 - GitHub, accessed March 15, 2026, https://github.com/ml-explore/mlx/discussions/3134
19. Benchmarked 11 MLX models on M3 Ultra — here's which ones are actually smart and fast, accessed March 15, 2026, https://www.reddit.com/r/LocalLLaMA/comments/1rkcvqa/benchmarked_11_mlx_models_on_m3_ultra_heres_which/
20. Where can I access deepseek-r1-distill-qwen-32b model - GMI Cloud, accessed March 15, 2026, https://www.gmicloud.ai/blog/deepseek-r1-distill-qwen-32b-access-deployment-and-high-performance-on-gmi-cloud
21. DeepSeek-R1 32B: Specifications and GPU VRAM Requirements - ApX Machine Learning, accessed March 15, 2026, https://apxml.com/models/deepseek-r1-32b
22. DeepSeek-R1-Distill-Qwen-32B - Model Info, Parameters, Benchmarks - SiliconFlow", accessed March 15, 2026, https://www.siliconflow.com/models/deepseek-r1-distill-qwen-32b
23. Deepseek R1 Distilled Models MMLU Pro Benchmarks : r/LocalLLaMA - Reddit, accessed March 15, 2026, https://www.reddit.com/r/LocalLLaMA/comments/1iserf9/deepseek_r1_distilled_models_mmlu_pro_benchmarks/
24. Qwen3-30B-A3B: Specifications and GPU VRAM Requirements - ApX Machine Learning, accessed March 15, 2026, https://apxml.com/models/qwen3-30b-a3b
25. DeepSeek-R1-Distill-Qwen-1.5B: The best small-sized LLM? | by Mehul Gupta - Medium, accessed March 15, 2026, https://medium.com/data-science-in-your-pocket/deepseek-r1-distill-qwen-1-5b-the-best-small-sized-llm-14eee304d94b
26. Run Qwen3 Next 80B A3B on macOS: Step-by-Step 2025 Guide - Codersera, accessed March 15, 2026, https://codersera.com/blog/run-qwen3-next-80b-a3b-on-macos-step-by-step-2025-guide
27. Systematic inference benchmarks: 5 models × 6 quants × 7 context lengths on M3 Ultra · ml-explore mlx · Discussion #3209 - GitHub, accessed March 15, 2026, https://github.com/ml-explore/mlx/discussions/3209
28. Research: vllm-mlx on Apple Silicon achieves 21% to 87% higher throughput than llama.cpp : r/LocalLLaMA - Reddit, accessed March 15, 2026, https://www.reddit.com/r/LocalLLaMA/comments/1qssxhx/research_vllmmlx_on_apple_silicon_achieves_21_to/
29. DeepSeek-R1-Distill-Qwen-32B is straight SOTA, delivering more than GPT4o-level LLM for local use without any limits or restrictions! : r/LocalLLaMA - Reddit, accessed March 15, 2026,

https://www.reddit.com/r/LocalLLaMA/comments/1i5s2yd/deepseekr1distillqwen32b_is_straight_sota/
30. Gemini API Free Tier 2026: Complete Guide to Rate Limits, Models & Getting Started, accessed March 15, 2026, https://blog.laozhang.ai/en/posts/gemini-api-free-tier
31. Gemini API Rate Limits 2026: Complete Per-Tier Guide with All Models, accessed March 15, 2026, https://aifreeapi.com/en/posts/gemini-api-rate-limits-per-tier
32. Gemini API Free Tier Rate Limits: Complete Guide for 2026, accessed March 15, 2026, https://aifreeapi.com/en/posts/gemini-api-free-tier-rate-limits
33. Gemini 3.1 Pro API Pricing & Performance: The Complete 2026 Guide for Developers, accessed March 15, 2026, https://www.glbgpt.com/hub/gemini-3-1-pro-api-pricing-performance-the-complete-guide-for-developers/
34. AI Hybrid WCAG Audits Expected in Q1 of 2027 | Accessible.org, accessed March 15, 2026, https://accessible.org/ai-hybrid-wcag-audits/
35. Leveraging Large Language Models for Sustainable and Inclusive Web Accessibility - MDPI, accessed March 15, 2026, https://www.mdpi.com/2504-2289/9/10/247
36. Qwen3.5-122B-A10B: Specifications and GPU VRAM Requirements, accessed March 15, 2026, https://apxml.com/models/qwen35-122b-a10b
37. Rate Limits - GroqDocs - Groq Console, accessed March 15, 2026, https://console.groq.com/docs/rate-limits
38. Groq On-Demand Pricing for Tokens-as-a-Service, accessed March 15, 2026, https://groq.com/pricing
39. Cerebras, accessed March 15, 2026, https://www.cerebras.ai/
40. Pricing - Cerebras, accessed March 15, 2026, https://www.cerebras.ai/pricing
41. API Rate Limits | Configure Usage Limits in OpenRouter, accessed March 15, 2026, https://openrouter.ai/docs/api/reference/limits
42. OpenRouter FAQ | Developer Documentation, accessed March 15, 2026, https://openrouter.ai/docs/faq
43. Gemini 2.5 Pro Free Tier Daily Quota: 100 RPD Explained (2026), accessed March 15, 2026, https://www.aifreeapi.com/en/posts/gemini-2-5-pro-free-tier-daily-quota-rpd
44. 2026 reality check: Are local LLMs on Apple Silicon legitimately as good (or better) than paid online models yet? - Reddit, accessed March 15, 2026, https://www.reddit.com/r/LocalLLM/comments/1ro3zwu/2026_reality_check_are_local_llms_on_apple/
45. Roo Code: A Guide With 7 Practical Examples - DataCamp, accessed March 15, 2026, https://www.datacamp.com/tutorial/roo-code
46. Qwen3.5:35b on Apple Silicon: How I Got 2x Faster Inference by Switching from Ollama to MLX (with benchmarks) : r/LocalLLaMA - Reddit, accessed March 15, 2026, https://www.reddit.com/r/LocalLLaMA/comments/1rezq19/qwen3535b_on_apple_silicon_how_i_got_2x_faster/
47. [Benchmark] Qwen3.5-122B-A10B FP8 weights / bf16 KV on 8x RTX PRO 6000

(SM120), accessed March 15, 2026, https://github.com/sgl-project/sglang/issues/19603
48. The Same Router, Better Backend: Multi-Model Routing with LM Studio and Apple's MLX, accessed March 15, 2026, https://medium.com/@michael.hannecke/the-same-router-better-backend-multi-model-routing-with-lm-studio-and-apples-mlx-78f53b2aabbb
49. OpenRouter Pricing Calculator & Cost Guide (Mar 2026) - CostGoat, accessed March 15, 2026, https://costgoat.com/pricing/openrouter
50. What a Robust Digital Accessibility Strategy Must Include in 2026, accessed March 15, 2026, https://www.pivotalaccessibility.com/2026/01/what-a-robust-digital-accessibility-strategy-must-include-in-2026/
51. Claude Opus 4.1 vs Gemini 3 Pro Preview - Pricing & Benchmark Comparison 2026, accessed March 15, 2026, https://pricepertoken.com/compare/anthropic-claude-opus-4.1-vs-google-gemini-3-pro-preview
52. Anthropic: Claude Opus 4.1 Review — Pricing, Benchmarks & Capabilities (2026), accessed March 15, 2026, https://designforonline.com/ai-models/anthropic-claude-opus-4-1/
53. AI API Pricing Comparison (2026): Grok vs Gemini vs GPT-4o vs Claude | IntuitionLabs, accessed March 15, 2026, https://intuitionlabs.ai/articles/ai-api-pricing-comparison-grok-gemini-openai-claude
54. Claude Sonnet 4 vs Qwen3 Next 80B A3B Instruct (Comparative Analysis) - Galaxy.ai Blog, accessed March 15, 2026, https://blog.galaxy.ai/compare/claude-sonnet-4-vs-qwen3-next-80b-a3b-instruct
55. MiniMax M2.5 vs GPT-5.2 vs Claude Opus 4.6 vs Gemini 3.1 Pro - Clarifai, accessed March 15, 2026, https://www.clarifai.com/blog/minimax-m2.5-vs-gpt-5.2-vs-claude-opus-4.6-vs-gemini-3.1-pro
56. Best practices for coding with agents - Cursor, accessed March 15, 2026, https://cursor.com/blog/agent-best-practices
57. Qwen3-Coder-Next: How to Run Locally | Unsloth Documentation, accessed March 15, 2026, https://unsloth.ai/docs/models/qwen3-coder-next
58. OpenAI Compatibility Endpoints | LM Studio Docs, accessed March 15, 2026, https://lmstudio.ai/docs/developer/openai-compat
59. Commands should allow model selection - Allow `model` property in command frontmatter - Feature Requests - Cursor - Community Forum, accessed March 15, 2026, https://forum.cursor.com/t/commands-should-allow-model-selection-allow-model-property-in-command-frontmatter/151140
60. Roo Code Marketplace | Roo Code Documentation, accessed March 15, 2026, https://docs.roocode.com/features/marketplace
61. Using Modes | Roo Code Documentation, accessed March 15, 2026,

https://docs.roocode.com/basic-usage/using-modes